Hidden Markov models for predicting stock market returns

R. Ahrensberg, S. Clemensen, A. Roelshøj, and A. G. Wolstrup

Niels Bohr Institute, University of Copenhagen
Submitted March 24, 2026

Abstract

Hidden Markov models (HMM) offer a powerful framework for inferring information about a hidden state space from a sequence of observed events. In this project, we apply a Gaussian HMM to predict stock prices of Novo Nordisk (NVO) using the features log returns, volatility, and momentum. We evaluate three training strategies: training on the NVO stock itself, on the S&P 500 index, and on a set of comparable pharmaceutical companies. We find comparable performance across them, with minor differences in error metrics. A simple trading strategy based on the model's predictions is also tested. Overall, the model captures short-term trends but remains limited by the Markov assumption, making it most suitable as part of a broader modeling approach.

1 Introduction

A hidden Markov model (HMM) is an augmentation of a Markov chain. A Markov chain is useful for modeling phenomena where the transition probabilities between states depend only on the current state, and not the entire sequence leading up to the current state. In many realistic cases, however, the states of interest are hidden. To address this problem we introduce the notion of HMMs, since they make it possible to infer the probability of the underlying sequence of hidden states based solely on the observed events. In this project, we develop a model using HMMs to predict stock closing prices of the Novo Nordisk stock (NVO) using the `HMMLearn` [1] package for Python. We evaluate three distinct training strategies: (i) training on the target stock itself, (ii) training on the S&P 500 index, and (iii) training on the stocks from a set of comparable pharmaceutical companies. The performance of these models is compared using mean absolute percentage error, root mean square error, and directional accuracy. Finally, we implement a trading strategy based on the model's predictions and assess its performance.

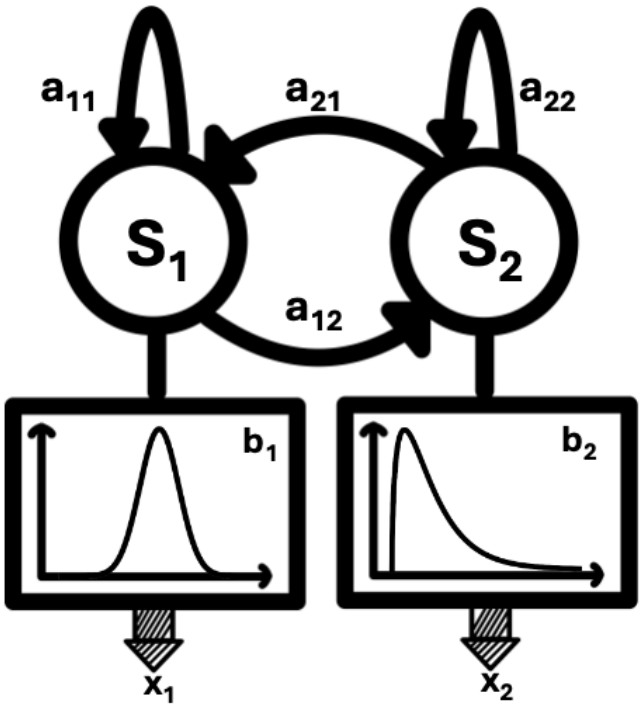

Figure 1: An HMM with continuous emissions. When in hidden state S_1 (S_2) a random number x_1 (x_2) is sampled from the distribution b_1 (b_2) as the emitted observation. The PDF for emissions from S_1 is a Gaussian with $\mu = 0$ and $\sigma^2 = 1$ while for S_2 it is the χ^2 distribution with $df = 3$.

2 Theory and method

2.1 Hidden Markov models

In an HMM, there are two linked processes consisting of a hidden sequence of states that evolves according to transition probabilities, and a sequence of observations generated from those states according to emission probabilities [2].

Formally, an HMM, λ, consists of an initial state sequence π, a transition matrix A, and an emission matrix B, and is denoted $\lambda = (\pi, A, B)$. The transition matrix describes the probabilities associated with transitions between hidden states as in a standard Markov chain, while the emission matrix describes probabilities of seeing each observation given each hidden state [2].

An HMM can have both discrete and continuous observed states. For the discrete case with n hidden states and m discrete observations, each entry b_{ij} in the emission matrix $B_{n\times m}$ is the probability of the hidden state i emitting the observation j. Due to normalization each row of $B_{n\times m}$ must sum to unity. For the continuous case with n discrete hidden states, the emission matrix becomes an n-dimensional column vector B_n. Here, each row is a probability density function (PDF). Importantly, each row can be a different PDF. In principle, any PDF can be used, but in our implementation we use a Gaussian distribution. An illustration of an HMM with continuous emissions is provided in Fig. 1. For Gaussian

emissions, the mean, μ, and the variance, σ^2, define the probabilities of distinct emissions. Any PDF is by definition normalized, and thus each row of B_n integrates to unity.

In real applications, HMMs have proven especially useful for speech recognition. Such a model can be trained by providing a vocabulary of words along with a set of examples of the words being spoken. Thus the audio signal is the observation, and the actual word or sentence is the hidden state to be determined. Similar models can be constructed for numerous other applications [2, 3, 4].

2.2 Classic HMM Algorithms

For HMMs, the main problem to solve is: Given a sequence of observations, O, determine the sequence of hidden states, Q. This task is broken down into three problems [2]:

1. (Likelihood) Given an HMM, $\lambda = (\pi, A, B)$, and an observation sequence O, what is the probability that O was produced by λ?

2. (Decoding) Given an observation sequence, O, and an HMM, $\lambda = (\pi, A, B)$, what is the most probable hidden state sequence, Q?

3. (Learning) Given an observation sequence, O, and the states of the HMM, learn the parameters $\pi, A,$ and B.

In most use-cases, the problems are solved from 3) to 1).

There are three algorithms dedicated to solving the aforementioned problems, known as the Forward-Backward algorithm (Likelihood), the Viterbi algorithm (Decoding) and the Baum-Welch algorithm (Learning) [2], which we will briefly explain.

The likelihood step is simply to calculate, given λ and an observation sequence $O = o_1, o_2, \ldots, o_T$, the likelihood $P(O|\lambda)$. The Forward-Backward algorithm is a dynamic programming procedure that recursively calculates the probability of being in hidden state j after seeing the first t observations, given λ. This is done by summing the probabilities of all the hidden sequences that can lead to the observed sequence. Thus, the probability of being in the state j at step t is

$$\alpha_t(j) = \sum_{i=1}^{N} \alpha_{t-1}(i) a_{ij} b_j(o_t), \tag{1}$$

where a_{ij} is the transition probability from state i to state j, and $b_j(o_t)$ is the probability of observing o_t given state j [2].

Similarly to the Forward-Backward algorithm, the decoding problem is solved by storing intermediate values using the Viterbi algorithm. At step t, the value

$$v_t(j) = \max_{i=1}^{N} v_{t-1}(i) a_{ij} b_j(o_t) \tag{2}$$

is computed. At the final step, $t = T$, this will result in the probability of the most likely sequence $Q = \max_{i=1}^{N} v_T(i)$. The most likely path can then be reconstructed by walking backwards through the hidden states by choosing the index for $v_t(j)$ in Eq. 2. Then the associated probability $P(Q \cup O)$ can be determined, that is, the probability of having both the observation and the state sequence. Together with step 1, this produces the desired probability $P(Q|O) = P(Q \cup O)/P(O)$ [2].

Both of these algorithms assume that the matrices A and B are known. They are, however, typically not available in advance, which is why these algorithms are usually performed last.

The learning step is done using the Baum-Welch algorithm, which estimates A, B, and π by adjusting these parameters to maximize the probability of the observation sequence given λ. It is an iterative algorithm, which computes probabilities from the initial state, and repeats until the probabilities converge [2].

The central object we need for this computation is the probability of being in state i at time t and state j at time $t+1$, given the observation sequence and the model:

$$\xi(i,j) = P(q_t = i, q_{t+1} = j | O, \lambda) \tag{3}$$

$$= \frac{\alpha_t(i)\, a_{ij}\, b_j(o_{t+1})\, \beta_{t+1}(j)}{\sum_{j=1}^{N} \alpha_t(j) \beta_t(j)}. \tag{4}$$

Here, $\beta_t(j)$ is the backward probability. It is the probability of seeing a sequence of observations given that the system is in state j:

$$\beta_t(j) = P(o_{t+1}, o_{t+2}, \ldots, o_T | q_t = j, \lambda). \tag{5}$$

It is found in much the same way as the forward probability (Eq. 1), but is done recursively backwards. The matrix elements a_{ij} of A and $b_j(o_t)$ are found in terms of the ξ's and iteratively re-estimated, forming the core of the algorithm [2].

Although this should in principle work unsupervised, it is in practice very sensitive to the initial conditions. Therefore, the structure of the HMM is often given beforehand, i.e. we set up the chain and states to be considered, like giving a vocabulary to the speech recognition example mentioned earlier.

2.3 HMM for stock prediction

The HMM categorizes a system into a discrete set of hidden states. These states are not necessarily well defined or understood. When we consider the stock market, these states may loosely be interpreted to describe the market as 'bearish', 'bullish', 'stagnant', etc., that is any hidden market state that characterizes the observed prices. The model, of course, does not assign semantic meaning to them. Instead, the states correspond to latent structures in the data that maximize the likelihood of the observed sequences.

To assist our task, we use the package `HMMLearn`. Here, the number of hidden states is a model hyperparameter, and just like in any other machine learning algorithm they can be tuned to improve the model. It is important to remember not to overtrain the model. This can be handled by splitting data into training data and testing data. The performance of the HMM can then be evaluated before it is used on actual data.

Since we want to not just characterize the market qualitatively, but to make quantitative predictions of future prices, we use the class `GaussianHMM` to model the emissions as normally distributed. This is done to map the discrete states into a continuous output. The goal of the training is to learn the means and variances of these Gaussian emissions to predict future emissions.

Most importantly, we don't directly try to predict stock prices, but the natural logarithm of the returns (log returns) defined as $\ln(P_t/P_{t-1})$, where P_t is the closing price at time t. Stock prices are inherently not Gaussian because they are non-negative, and since our model assumes Gaussian emissions, directly predicting stock prices would violate our model. It turns out that the stock price distribution can be modeled as log-normal, i.e. the natural logarithm of the price is assumed normally distributed. However, prices of different stocks cannot be compared in absolute terms, since the price level can vary widely between stocks. This is the reason to use log-returns instead of prices, since returns should be invariant across different stocks [5]. Using the model to predict log returns, we then transform our predictions back to stock prices to quantitatively assess our model.

2.4 Algorithm for Stock prediction

With this theoretical framework in place, we now describe how the model is implemented for stock prediction.

Our prediction algorithm uses the Baum-Welch algorithm to train an HMM with Gaussian emissions. We train the HMM on log returns, volatility, and momentum, all features derived from the stock price. Training on these features allows the model to be quite general in modeling most types of stochastic phenomena, and not just stock prices. Volatility is computed as the rolling standard deviation of closing prices over the past ten time units (e.g. days), while momentum is defined as the percentage change between the current closing price and the price five units of time prior. To avoid one of the features dominating during training, we use Scitkit-Learn's `StandardScaler` [6] to rescale the features before training. Once completed, we save the means and standard deviations of each feature for each hidden state. These serve to characterize the underlying hidden states which determine the emissions, which are the stock prices in our case.

For each time step t in the test data, a rolling window of past observations up to time $t-1$ is used with the Viterbi algorithm to infer the hidden state at $t-1$. Then the transition matrix is used to determine the most likely hidden state at time t. In the event of a tie, one of the tied states is selected at random. Once the hidden state at t is determined, we sample a log return value r_t from the Gaussian distribution associated with that state, using the learned mean μ and variance σ^2 of the state saved earlier. The predicted closing price $\hat{P}_t$ is then computed from the previous closing price P_{t-1} using the sampled return $\hat{P}_t = P_{t-1} \cdot \exp(r_t)$. By iteratively applying this procedure across the test set, the algorithm generates a sequence of predicted stock prices.

3 Data

Our source of data is stock prices from Yahoo Finance, which we implement in our program using the `yfinance` package for [7]. The package allows us to pull historical closing stock prices on either a daily or monthly level. The data we aim to predict is the daily closing price of the Novo Nordisk (NVO) stock.

We retrieve data points for four years from 2022-01-01 to 2025-12-31. We train our model using three different training setups: (i) training on the NVO stock itself, (ii) training on stocks from a set of comparable pharmaceutical companies using Eli Lilly, Pfizer, Johnson & Johnson, Merck & Co., AstraZeneca and Sanofi - and (iii) training on the S&P 500 market index consisting of the 500 US stocks with the largest market capitalization. For all training subsets, we train on data from 2022-01-01 to 2023-12-29, amounting to 491 training days, and then we predict from 2024-01-02 up to 2025-12-31.

To evaluate the model, we calculate the mean absolute percentage error (MAPE), root mean square error (RMSE) and directional accuracy (DA). MAPE is defined as

$$\frac{100}{N}\sum_{t=1}^{N}\left|\frac{y_t-\hat{y}_t}{y_t}\right|, \tag{6}$$

RMSE as

$$\sqrt{\frac{1}{N}\sum_{t=1}^{N}(y_t-\hat{y}_t)^2}, \tag{7}$$

and DA as

$$\frac{1}{N-1}\sum_{t=2}^{N}\mathbf{1}\Big[\operatorname{sgn}(y_t-y_{t-1})=\operatorname{sgn}(\hat{y}_t-\hat{y}_{t-1})\Big], \tag{8}$$

where y_t is the observed value at time t, $\hat{y}_t$ the predicted value, N the total amount of observations, and $\mathbf{1}[\cdot]$ the indicator function, i.e. it returns 1 if the argument is true and 0 if not.

4 Results

To prevent overtraining the model, we used `Optuna` [8], a package to optimize model hyperparameters. We use it to minimize $\mathrm{MAPE}(\mathbf{y},\hat{\mathbf{y}})+\mathrm{RMSE}(\mathbf{y},\hat{\mathbf{y}})-\mathrm{DA}(\mathbf{y},\hat{\mathbf{y}})$ with respect to the number of hidden states the model is given and the size of the rolling window. The model aims for low errors in MAPE and RMSE, while aiming for high DA. The optimal solution is 7 hidden states and a window size of 30 days.

In Tab. 1 we show how each training setup scored in terms of the evaluation metrics. The results are for predicting the entire two-year period from 2024-01-02 to 2025-12-31. For clarity, we have plotted the last 100 predicted closing prices for our three training setups and compared them to the actual closing price in Fig. 2.

Training Setup	MAPE (%)	RMSE (USD)	DA (%)
(i) NVO	0.0242	2.9475	47.70
(ii) S&P 500	0.0229	2.7303	48.30
(iii) Multiple	0.0258	3.0478	49.30

Table 1: Evaluation Metrics for different training samples.

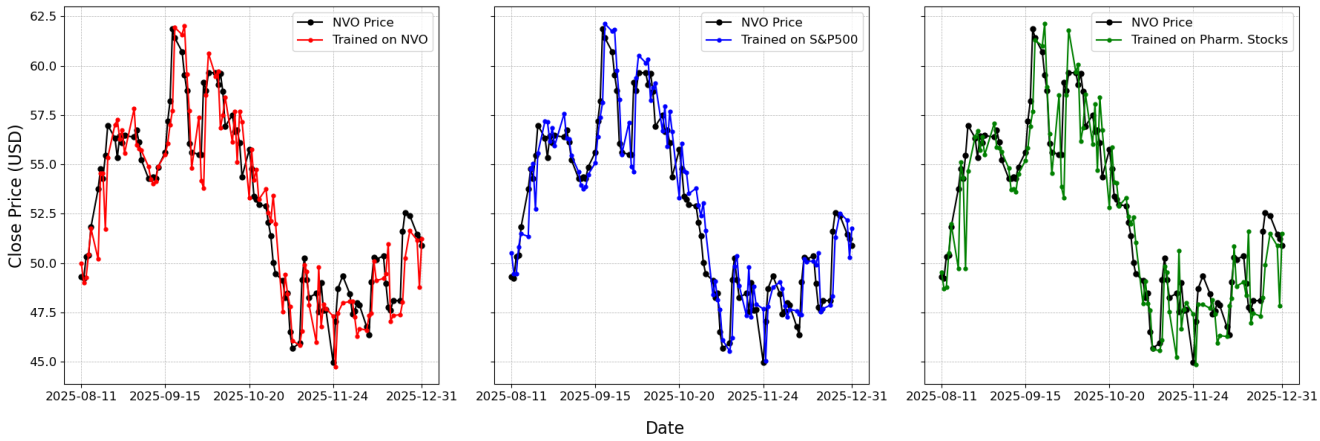

Figure 2: The last 100 (days since 2025-08-11) values of predicted closing price for our three training setups, i.e. trained on NVO (red), on S&P 500 (blue) and on a set of pharmaceutical stocks (green), compared to the observed closing price (black).

4.1 Trading strategy based on predictions

In order to test the applicability of the model, we attempt to implement a trading strategy based on our model's predictions. This involves identifications which are not done by the HMM itself, so it shall serve as a simple proof of concept.

For each time step in the real and predicted data set, we produce a 'buy' or 'sell' signal based on the predicted price movement and market classification, i.e. bullish or bearish. The market is classified as bullish or bearish according to two criteria: 1) Is the predicted price higher or lower than the current price and 2) Are we currently in an uptrend or downtrend? The latter is identified using exponentially weighted moving averages over recent days, which place greater weight on more recent observations than on earlier ones. We use moving averages of 3 and 6 days respectively to capture trends of different lengths.

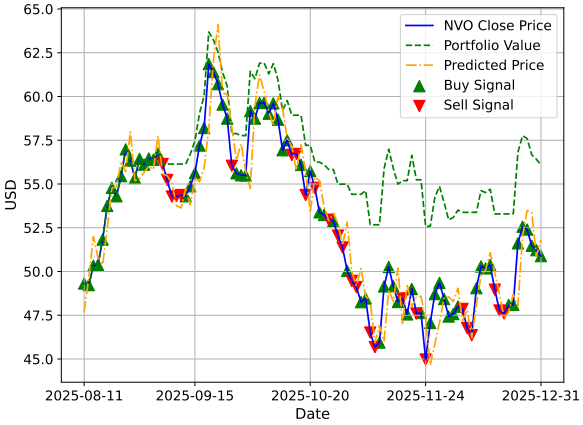

Figure 3: Trading strategy trained on NVO and implemented on NVO 2025-08-11 to 2025-12-31

Based on these buy/sell signals, we can track the portfolio value in the case that one simply owns either 0 or 1 units of stock at any point in time, and acts according to the signals. The result of applying this to daily NVO data from 2025-08-11 to 2025-12-31 is seen on Fig. 3.

As can be seen in Fig. 3, the trading strategy is far from perfect, but it is able to use the predictions to maintain a higher value than if one simply held onto the asset by reducing losses when the price declines. This suggests that with the right tweaking of the signal generation process, the model's predictions should be good enough to be implemented in a viable trading strategy.

5 Discussion

Our main results are presented in Tab. 1. Overall, performance is similar across the three training strategies, with a slight advantage for the model trained on the S&P 500 index. All models achieve low MAPE values, indicating good predictive accuracy in terms of magnitude. The average closing price of the NVO stock during the prediction period is 91.84 USD. Given RMSE values of approximately 3 USD across all setups, this corresponds to a relative error of about 3%, which suggests reasonably accurate closing price predictions. However, all models perform poorly in terms of DA, with values below 50%. This indicates that the predicted direction of price movement is incorrect more often than correct. For comparison, the stock price increases 52.14% of the time in the training data. A naive strategy that always predicts an increase would achieve an accuracy of 47.81% on the prediction period, which is lower than the DA obtained by training setups (ii) and (iii).

As the name implies, HMMs work on a Markovian assumption - that state transition probabilities only depend on the current state of the system. This presents some limitations. With our model, we only make predictions one time step ahead. We have tried predicting further, but when applying the model to its own prediction, it simply amplifies that prediction - for example, a price predicted to rise at step $t+1$ will then be predicted to keep rising further at $t+2$. This would apply to any stochastic variable one could try to predict in this way, and is rooted in the Markovian assumption. This, of course, makes it difficult to implement any robust trading strategy based solely on HMMs. In order to obtain long term predictions, one could instead sample monthly data and try to predict next month. However, this comes at the price of throwing away most of the available data, and furthermore, the trends that we want the model to learn may be inherently more short-term, not to mention discarding important trading opportunities. The impact of these concerns may depend on the system being modeled, but in general, we obtained the best results using daily data.

6 Outlook

Our model could be expanded on by implementing uncertainties or confidence intervals on our daily predictions. Currently, emission probabilities are sampled from Gaussian distributions. Using a sample and not simply the mean emulates the stochasticity of the underlying data. However, a similar model could instead use the mean and standard deviation of the distribution to provide information about the uncertainty on each prediction. This way, one could obtain approximate probability of each movement direction rather than the precise price, which may be more useful in some cases. Furthermore, one could presumably use the likelihood algorithm to compare different possible states, but we did not pursue this.

There are obvious limitations to the predictive powers of HMMs alone regarding stock prices, since the Markov assumption might not always be satisfied and due to the volatile nature of markets. Instead of using HMMs to directly predict stock prices, HMMs have been shown to be very effective in characterizing market regimes [9], and can in combination with other models be very powerful, e.g. Long Short-Term Memory [10].

7 Conclusion

In this project, we have explored the use of hidden Markov models for predicting stock market behavior, focusing on the Novo Nordisk (NVO) stock over the period from 2024-01-01 to 2025-12-31. By modeling log returns, volatility, and momentum using a Gaussian HMM, we constructed a model capable of identifying latent market states and producing short-term predictions of future prices.

We evaluated three different training strategies and found that the overall performance of the model is similar across these setups. Training on the S&P 500 index yields slightly lower prediction errors, while training on multiple related stocks provides a marginal improvement in directional accuracy. This indicates that different training data can influence specific aspects of the model performance, although no single approach is clearly superior in all metrics. The model is, however, limited by the Markov assumption, which implies that predictions depend only on the current state. As a result, the model performs best for short-term forecasting, while longer-term predictions become unreliable.

In summary, hidden Markov models offer a useful probabilistic framework for modeling financial time series and capturing underlying structure in the data. While the predictive performance is reasonable, the limitations of the model suggest that it is better suited to be used in combination with more advanced models rather than as a standalone tool for stock market prediction.

References

[1] HMMLearn Contributers. HMMLearn. https://hmmlearn.readthedocs.io/en/latest/. Accessed: 24 March 2026. 2010.

[2] L. R. Rabiner et al. "An introduction to hidden Markov models". In: IEEE ASSP Magazine 3 (1986), pp. 4–16. URL: https://api.semanticscholar.org/CorpusID:11358505.

[3] A. Kundu et al. "Recognition of Handwritten Word: First and Second Order Hidden Markov Model Based Approach". In: Pattern Recognition 22.3 (1989), pp. 283–297.

[4] I. R. Sipos et al. "Parallel Optimization of Sparse Portfolios with AR-HMMs". In: Computational Economics 49 (2017), pp. 563–578. DOI: 10.1007/s10614-016-9579-y. URL: https://doi.org/10.1007/s10614-016-9579-y.

[5] K. Zucchi. "Understanding Lognormal vs. Normal Distributions in Financial Analysis". In: Investopedia (2025). URL: https://www.investopedia.com/articles/investing/102014/lognormal-and-normal-distribution.asp.

[6] Scikit-Learn Contributors. Scikit-Learn. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html. Accessed: 24 March 2026. 2007-2025.

[7] R. Aroussi. yfinance. https://pypi.org/project/yfinance/. Version 1.2.0. Apache Software License. Not affiliated with Yahoo, Inc. Accessed: 24 March 2026. 2019.

[8] Optuna Contributors. Optuna Documentation. https://optuna.readthedocs.io/en/stable/. Accessed: 24 March 2026. 2018.

[9] D. Borst. "Detecting Market Regimes: Hidden Markov Model". In: Medium (2025). URL: https://datadave1.medium.com/detecting-market-regimes-hidden-markov-model-2462e819c72e.

[10] J. Jiang et al. "Forecasting movements of stock time series based on hidden state guided deep learning approach". In: Information Processing Management 60.3 (2023), p. 103328. ISSN: 0306-4573. DOI: https://doi.org/10.1016/j.ipm.2023.103328. URL: https://www.sciencedirect.com/science/article/pii/S0306457323000651.